



API-ІНТЕРФЕЙСИ ДЛЯ РОЗПОДІЛУ НАКЛАДНИХ ВИТРАТ ПО SKU

Технічна документація

Версія: 1.0

Дата: 14 липня 2025

ЗМІСТ

Загальна інформація	2
Функціональні вимоги.....	2
Аналіз рішення.....	3
Модель даних для тестування	3
Архітектура рішення (Clean Architecture)	3
Рішення	4
Схема взаємодії компонентів (на прикладі запиту на звіт «Маржа по SKU»)	4
Посилання на репозиторій	5
API Документація.....	5

ЗАГАЛЬНА ІНФОРМАЦІЯ

Фінансовий відділ формує звіт P&L, який включає ключові показники: доходи від продажів, собівартість реалізованої продукції, валовий прибуток. Цей звіт є зведеним і не деталізується до рівня номенклатури (SKU).

У якості джерела для розрахунку фінансових показників використовується звіт з продажів, що містить деталізацію в розрізі проєктів, клієнтів та товарних позицій.

Окремо ведеться звіт по накладних витратах, які фіксуються на рівні статей витрат та проєктів, без розподілу по конкретних замовленнях чи товарах.

ФУНКЦІОНАЛЬНІ ВИМОГИ

Необхідно реалізувати механізм розподілу накладних витрат на рівень товарних позицій (SKU). Розподіл має виконуватись у розрізі проєктів, пропорційно обсягу продажів кожного SKU у межах відповідного проєкту.

Очікуваний результат - звіт "**Маржа по SKU**", який повинен містити усі необхідні дані з вихідного звіту про продажі, а також додатково обчислені показники:

Параметри та атрибути звіту "Маржа по SKU"

- Period (дата)
- Project (проєкт)
- Customer (клієнт)
- SKU (товарна позиція)
- Order number (номер замовлення)
- Quantity (кількість)
- Sales summa (сума продажу)
- Summa cost of goods (сума собівартості)

Обчислювані поля:

- Gross profit (Валовий прибуток)
- Overhead costs (Накладні витрати)
- Profit (Прибуток)

Звіт повинен бути придатним для подальшої інтеграції з Power BI та для публікації на внутрішньому корпоративному сайті.

АНАЛІЗ РІШЕННЯ.

З огляду на потенційно великі обсяги даних щодо продажів та накладних витрат, розрахунок розподілу витрат доцільно реалізовувати на рівні бази даних SQL.

Такий підхід забезпечує ефективну обробку інформації без зайвого навантаження на рівень аплікації.

Розподіл може бути реалізований шляхом stored procedure або view.

Для подальшої інтеграції з Power BI та внутрішнім корпоративним порталом передбачається розробка вебсервісів, які надаватимуть результат у форматі JSON або CSV:

- розподіл витрат,
- довідники (товари, проекти, клієнти, накладні витрати)

МОДЕЛЬ ДАНИХ ДЛЯ ТЕСТУВАННЯ

Для реалізації логіки та тестування рішення створено окреме середовище в SQL, що імітує роботу ERP-системи.

Тестові таблиці наповнені даними, що дозволяють перевірити правильність логіки розподілення витрат та побудови маржинального звіту.

Структура даних:

- Довідники:
 - Товари (SKU)
 - Клієнти
 - Проекти
 - Накладні витрати
- Фактичні дані (операції продажу та реєстрації витрат):
 - Продажі у розрізі SKU, клієнтів та проектів
 - Накладні витрати у розрізі проектів

АРХІТЕКТУРА РІШЕННЯ (CLEAN ARCHITECTURE)

В проєкті застосовано Clean Architecture у версії моноліт, що забезпечує високу підтримуваність і масштабованість системи:

- **Domain:** Основні сутності: SKU, Customer, Project, Margin.
- **Application:** рівень бізнес-логіки
- **Infrastructure:** Робота з БД (репозиторій на Dapper)
- **Presentation:** Контролери ASP.NET Web API.

Реалізація розподілу витрат:

Побудовано SQL view MarginSKU, яке реалізує логіку розподілу.
Застосовано CTE (Common Table Expressions) для поетапної обробки:

- Обчислення загального обсягу продажів по проекту.
- Визначення частки кожного SKU у загальному обсязі.
- Пропорційне розподілення накладних витрат на SKU.

API-рівень (ASP.NET Web API)

Реалізовані вебсервіси, що забезпечують інтеграцію з аналітичними системами, такими як Power BI або внутрішній корпоративний портал. Дані надаються у форматі JSON або CSV.

Для всіх контролерів додано базовий контролер BaseApiController, створений на основі ControllerBase.

В ньому реалізовано логіку аналізу вихідного формату (JSON/CSV), серіалізацію CSV та валідацію дат.

СХЕМА ВЗАЄМОДІЇ КОМПОНЕНТІВ (НА ПРИКЛАДІ ЗАПИТУ НА ЗВІТ «МАРЖА ПО SKU»)

Запит на побудову звіту Маржа по SKU проходить наступні рівні:

Контролер MarginController. GetAllAsync verb [HttpGet]

Отримує HTTP-запит від зовнішньої системи (Power BI, браузер), валідує параметри дати «з» за «по».

Передає його у бізнес-логіку.

Виконує логіку формування відповіді (включаючи обробку формату CSV/JSON).

Сервіс бізнес-логіки MarginService.GetAllAsync

Викликає метод репозиторію для отримання оброблених даних

Репозиторій MarginRepository.GetAllAsync

Напрямку звертається до бази даних (через Dapper) та виконує запит до SQL view dbo.MarginSKU.

База даних SQL View: dbo.MarginSKU

Реалізує всю логіку розподілу накладних витрат за допомогою CTE.

API сервіси на .NET

<https://github.com/ValDynamo/SQL-Margin/tree/MarginSKU/API.MarginSKU>

SQL скрипт з тестовими даними і механізмом розподілення

<https://github.com/ValDynamo/SQL-Margin/blob/MarginSKU/DatabaseMargin2.sql>

Опис задачі розподілення для рішення в SQL

<https://github.com/ValDynamo/SQL-Margin/blob/MarginSKU/Margin%20SKU.pdf>

API ДОКУМЕНТАЦІЯ

- Всі ендпоінти підтримують необов'язковий параметр `format`, значення JSON або CSV
- Якщо параметр `format` не вказано, відповідь повертається у форматі JSON за замовчуванням.
- Для ендпоінтів, які повертають звітні дані з датами:
 - Параметри `fromDate` та `toDate` є обов'язковими.
 - Формат дат ISO 8601: `yyyy-MM-dd`.
 - При відсутності обов'язкових параметрів або некоректному форматі дат API повертає помилку 400 (Bad Request).

GET /api/margin

- Повертає звіт із розподіленими накладними витратами по SKU за вказаний період.
- Обов'язкові параметри:
 - **fromDate** - початкова дата (включно).
 - **toDate** - кінцева дата (включно).
- Необов'язковий параметр:
 - **format** - формат відповіді: CSV або JSON (за замовчуванням).
- Приклад запити:
 - `/api/margin?fromDate=2022-08-01&toDate=2022-09-01&format=json`

GET /api/goods

- Повертає довідник товарів (SKU), обов'язкових параметрів немає
- Приклад запити:
 - `/api/goods?format=csv`

GET /api/customers

- Повертає довідник клієнтів, обов'язкових параметрів немає
- Приклад запити:
 - `/api/customers`

GET /api/projects

- Повертає довідник проектів, обов'язкових параметрів немає
- Приклад запити:
 - `/api/projects?format=json`